Master Thesis

NL2Vis: Multi-agent System for Natural Language to Data Visualization

Jacob Chen
(matriculation number 1980440)

May 4, 2026

Submitted to
Social Data Science Group
Prof. Dr. Marc Ratkovic
University of Mannheim

Abstract

This thesis investigates the development of autonomous, multi-agent systems for the Natural Language to Visualization (NL2Vis) task, focusing on overcoming the limitations of existing specialized frameworks. Current state-of-the-art systems tend to excel in one of two key areas: sophisticated, multi-table database querying or high-fidelity, iterative visual refinement, but rarely both. Furthermore, these systems almost exclusively rely on costly, proprietary Large Language Models (LLMs). This work proposes *LocalVisAgent*, a novel hybrid multi-agent framework that integrates the robust database-processing architecture of `nvAgent` with a multimodal visual feedback loop inspired by `MatPlotAgent`. This entire framework is implemented using locally-run, open-source LLMs, including Llama 3 and LLaVA, to ensure cost-effectiveness and data privacy. We evaluate this hybrid agent against two distinct, specialized benchmarks: `VisEval`, to test complex database interaction, and `MatPlotBench`, to test visual and aesthetic accuracy. Our results demonstrate that `LocalVisAgent` successfully retains the strong database querying performance of its baseline while dramatically improving visual fidelity on complex plotting tasks. This work validates the efficacy of a hybrid, local-first approach, offering a viable and comprehensive solution that bridges the gap between database-centric and visualization-centric NL2Vis systems.

Contents

Abstract **ii**

1. Introduction **1**
1.1. Research Motivation . . . 1
1.2. Approach and Scope . . . 1
1.3. Contributions . . . 2

2. Literature Review **3**
2.1. The NL2Vis Pipeline and What Makes a Good Visualization . . . 3
2.2. From Rule-Based Systems to Large Language Models . . . 4
2.3. Multi-Agent LLM Frameworks for NL2Vis . . . 5
2.4. Open Challenges in Deploying NL2Vis Systems . . . 6
2.4.1. Ambiguity and Intent Underspecification . . . 6
2.4.2. Scalability and Context Constraints . . . 6
2.4.3. Privacy, Data Confidentiality, and the Cost of Proprietary APIs . 7
2.4.4. Evaluation and Human-in-the-Loop Feedback . . . 7
2.5. Open-Source LLMs and Local Deployment . . . 7
2.6. Evaluation Benchmarks: From nvBench to VisEval . . . 8
2.7. Summary and Research Gap . . . 9

3. Methodology **11**
3.1. Agent 1: Processor (Database Specialist) . . . 11
3.2. Agent 2: Composer (Visualization Planner) . . . 12
3.3. Agent 3: Synthesizer (Code Generator) . . . 12
3.4. Agent 4: Multimodal Critic (Iterative Refiner) . . . 12

4. Experimental Setup **14**
4.1. Models and Baselines . . . 14
4.2. Benchmarks . . . 14
4.3. Evaluation Metrics . . . 15

5. Results and Evaluation **16**
5.1. Results 1: Performance on VisEval (Database Task) . . . 16
5.2. Results 2: Performance on MatPlotBench (Visualization Task) . . . 16
5.3. Qualitative Analysis and Ablation . . . 17
5.4. Discussion . . . 18

6. Conclusion **19**
6.1. Limitations 19
6.2. Future Work 19

Bibliography **21**

A. Additional Experimental Results **23**

B. Ehrenwörtliche Erklärung **25**

1. Introduction

The ability to extract insights from data is a central driver of progress in science, policy, and industry. In practice, however, access to that insight is often limited by technical barriers: many domain experts can interpret results, but cannot easily produce them from raw relational data (Wu et al. 2024). Natural Language to Visualization (NL2Vis) aims to reduce this gap by allowing users to request visualizations in everyday language rather than through manual query writing or code authoring (Maddigan & Susnjak 2023).

An effective NL2Vis system must solve two tightly coupled sub-problems. First, it must perform robust **Text-to-SQL** translation, mapping user intent to correct queries over often complex, multi-table schemas (Zhu et al. 2024; Wu et al. 2024). Second, it must perform **Text-to-Code** generation, converting retrieved data into executable visualization code (for example in Python) that is both syntactically valid and semantically aligned with the user's request (Huynh & Lin 2025). Errors in either stage can invalidate the final chart.

Recent work has increasingly converged on LLM-based multi-agent systems for this pipeline (Li et al. 2024). A representative example is `nvAgent` (Ouyang et al. 2025), which decomposes the task into a Processor, Composer, and Validator and reports strong performance on `VisEval` (Chen et al. 2025). Yet this progress comes with a practical trade-off: leading systems still depend on proprietary models such as GPT-4o. For organizations handling sensitive data, this dependency is often infeasible.

1.1. Research Motivation

This thesis investigates a concrete practical question:

> *How far can locally-run, open-source LLMs take an `nvAgent`-style multi-agent NL2Vis system on realistic benchmark tasks?*

The motivation is twofold. First, privacy and governance constraints in enterprise and public-sector settings often prohibit sending schema context and query content to third-party APIs. Second, recent progress in open-source and code-specialized LLMs suggests that the performance gap to proprietary models may now be narrow enough to justify local deployment in many settings.

1.2. Approach and Scope

We use `nvAgent`'s architecture as the baseline and run a systematic model sweep with locally served open-source LLMs. The evaluated models range from 7B instruction-tuned

baselines (Qwen2.5-7B) to code-specialized variants (Qwen2.5-Coder 7B, 14B, and 32B) and a Mixture-of-Experts coder model (Qwen3-Coder-30B-A3B). All models are served through vLLM (Kwon et al. 2023) and quantized with AWQ where appropriate (Lin et al. 2024). Evaluation is conducted on `VisEval` (Chen et al. 2025), including both single-table and multi-table settings.

Beyond model scaling, we test two targeted optimizations: (i) a visual reviewer agent that critiques rendered plots, and (ii) prompt-level improvements to the Composer to strengthen alignment with the original user intent. The analysis shows that the reviewer does not improve pass rates in this setup; most failures are caused by incorrect data integration upstream, not by chart rendering quality.

1.3. Contributions

The main contributions of this thesis are:

1. A systematic empirical evaluation of open-source LLMs of different sizes and specializations within an `nvAgent`-style multi-agent NL2Vis pipeline.

2. A failure-mode analysis showing that data-integration errors, rather than code-generation errors, are the dominant bottleneck for smaller local models.

3. A practical deployment blueprint for privacy-preserving NL2Vis systems, including clear trade-offs between model capability, cost, and data-governance constraints.

2. Literature Review

Natural Language to Visualization (NL2Vis) sits at the intersection of natural language processing, data visualization, and human–computer interaction. At its core, the task asks a system to take a free-form query in everyday language and produce a chart or other visual representation from structured, relational data—without requiring the user to write code or formulate explicit queries (Wu et al. 2024). The appeal is clear: it democratizes data exploration, allowing business analysts, policymakers, and domain experts to interact with complex databases on their own terms. Yet the technical challenge is equally significant: the system must simultaneously resolve what the user means, retrieve the right data, and render an appropriate visualization—a pipeline that compounds errors across its stages.

This chapter reviews the methodological foundations and current state of the art in NL2Vis, with particular attention to the challenges and design decisions most relevant to this thesis. Our focus is on systems that use Large Language Models (LLMs) in multi-agent configurations, the practical constraints of deploying such systems with sensitive organizational data, and the benchmarks used to evaluate them. We structure the review around five themes: the evolution of NL2Vis methods, the emergence of multi-agent frameworks, the technical and practical challenges of real-world deployment, the viability of open-source local LLMs, and the benchmark landscape.

2.1. The NL2Vis Pipeline and What Makes a Good Visualization

The NL2Vis pipeline can be understood as the composition of two distinct sub-tasks. The first is **Text-to-SQL**: translating a natural language query into a SQL statement that retrieves the right data from a relational database (Zhu et al. 2024). This sub-task presents significant challenges of its own—the system must link tokens in the user's query to columns and tables in the schema, handle joins across multiple tables, and correctly infer aggregations and filters that the user has not made explicit. The second sub-task is **Text-to-Code**: taking the retrieved data and generating executable code—typically Python with Matplotlib or Plotly—that renders an appropriate chart (Huynh & Lin 2025). Errors at either stage compound: incorrect data retrieval produces a visually polished but semantically wrong chart, while faulty code generation fails execution entirely.

A good visualization is more than one that runs without error. It must faithfully reflect the user's analytical intent: the correct chart type, appropriate axis mappings, accurate aggregations, and readable labels (Maddigan & Susnjak 2023). Earlier work in

the field focused heavily on syntactic correctness—whether the output code compiles and renders—but the community has increasingly recognized that a chart can be syntactically valid and semantically wrong at the same time. A bar chart that displays aggregated sales by the wrong grouping dimension, or a line chart that inverts its temporal axis, is technically valid but analytically misleading. Defining and measuring visual quality rigorously remains an open problem, and evaluation methodology is itself an active area of research, as we discuss in Section 4.2.

2.2. From Rule-Based Systems to Large Language Models

Early NL2Vis research relied on hand-crafted grammars and semantic parsers. These systems would parse a query into structured slots—chart type, data columns, aggregation function—using carefully designed linguistic rules. They were interpretable and predictable within their designed scope, but brittleness under paraphrase and an inability to generalize beyond the rules' coverage severely limited their reach.

Neural sequence-to-sequence models changed the picture considerably. Treating NL2Vis as a machine translation problem—translating natural language into a visualization specification language—allowed systems like ncNet to learn mappings end-to-end from paired examples (Luo et al. 2022). ncNet introduced Transformer-based seq2seq models with visualization-aware training strategies, including attention forcing and chart-template conditioning, and demonstrated clear improvements over rule-based baselines. These models relied on the nvBench dataset (Luo et al. 2021), which provided the first large-scale paired corpus for the task and will be discussed in more detail in Section 4.2.

The rise of large language models shifted the paradigm again, and decisively. Rather than training specialized models on NL2Vis-specific data, recent work has exploited the strong code generation and instruction-following capabilities of LLMs through prompting and in-context learning. Chat2Vis demonstrated that prompting GPT-3 and ChatGPT could produce usable visualizations from natural language queries with minimal task-specific engineering (Maddigan & Susnjak 2023). Prompt4Vis extended this idea by combining example mining—retrieving semantically similar examples from a corpus—with schema filtering to reduce irrelevant context, achieving substantial gains through better in-context prompt construction (Li et al. 2025). LIDA proposed a grammar-agnostic pipeline driven entirely by LLMs, capable of producing visualizations in multiple output languages without being tied to any particular chart specification format (Dibia 2023). Together, these works established that modern LLMs, when appropriately prompted, can handle the reasoning demanded by NL2Vis without task-specific fine-tuning—at least in the single-table, clean-data regime.

The methodological shift toward LLMs has been substantial. The emergence of accessible LLM APIs in 2022 and 2023 catalyzed a rapid expansion of LLM-based approaches to NL2Vis, with systems increasingly relying on prompting and in-context learning rather than task-specific model training, a trend documented across multiple recent empirical studies (Wu et al. 2024; Maddigan & Susnjak 2023; Li et al. 2025).

2.3. Multi-Agent LLM Frameworks for NL2Vis

A single LLM handling the entire NL2Vis pipeline faces a fundamental tension: the skills required to process a complex multi-table SQL query are not the same as those needed to generate polished Matplotlib code, or to identify a visual error in a rendered chart. This observation has motivated a divide-and-conquer approach, where a multi-agent system assigns each stage of the pipeline to a specialized agent. LLM-based multi-agent frameworks have demonstrated strong performance in complex software engineering tasks generally (Li et al. 2024), and the NL2Vis community has adopted this paradigm quickly.

`nvAgent` (Ouyang et al. 2025) is the most directly relevant prior work for this thesis. It organizes the NL2Vis pipeline around three collaborating agents. The *Processor* agent handles the database interaction: given a user query and a database schema, it filters irrelevant tables and columns, classifies query complexity as single-table or multi-table, and generates an executable SQL query. This schema-filtering step is critical—it reduces the context size passed to downstream agents and helps the system focus on only the relevant portions of the schema. The *Composer* agent takes the Processor's output and generates a high-level visualization plan in a structured Visual Query Language (VQL) format, specifying chart type, axis mappings, and groupings before any code is written. The *Validator* agent translates this plan into executable Python code and runs it; if execution fails, the error message is fed back to the Composer for correction. Evaluated on the co-developed VisEval benchmark (Chen et al. 2025), nvAgent achieved pass rate improvements of 7.88% on single-table and 9.23% on multi-table queries over the best prior baseline, and maintained a roughly 20% margin over non-agentic approaches. The multi-table strength is particularly notable, as many earlier systems struggle when a visualization requires joining data across multiple tables.

Despite this strong database performance, nvAgent's Validator has an important limitation: it only detects code execution failures. A chart that runs successfully but displays the wrong data, uses an inappropriate chart type, or has missing labels is approved without question. This gap between syntactic success and semantic correctness is a central weakness, and one that motivates the visual feedback approaches in more recent systems.

`MatPlotAgent` (Yang et al. 2024) addressed this gap from the opposite direction, focusing on visual quality. Its architecture adds a multimodal feedback loop to the code generation pipeline: after a chart is rendered, a Vision Language Model (VLM) examines the image and provides natural language critique—noting, for example, that an axis label is missing or that the chart type does not match the request. This critique is fed back to the code generation module for iterative correction. MatPlotAgent demonstrated that even a small number of VLM-guided refinement iterations substantially improves visual fidelity on complex scientific visualization tasks.

`PlotGen` (Goswami et al. 2025) pushes this feedback idea further, decomposing the visual critique into three specialized agents: a Numeric Feedback Agent that verifies data values, a Lexical Feedback Agent that checks labels and titles, and a Visual Feedback Agent that assesses chart type and layout. This granular decomposition proved effective, yielding 4–6 percentage point improvements over strong baselines on complex visualization tasks. Similarly, Shi et al. trained a VLM specifically to judge visualization quality

and provide code-level repair suggestions, demonstrating that a model fine-tuned on examples of good and bad charts gives more actionable feedback than a general-purpose VLM used zero-shot (Shi et al. 2025).

These systems reveal a persistent pattern in the field: systems that excel at database querying (nvAgent) are weak at visual validation, and systems that produce high-quality visuals (MatPlotAgent, PlotGen) are typically demonstrated on pre-processed, single-file datasets that sidestep the database interaction entirely. No single framework has convincingly addressed both halves of the pipeline simultaneously. This *specialization gap* is one of the core architectural problems this thesis sets out to address.

2.4. Open Challenges in Deploying NL2Vis Systems

Beyond the specialization gap, the NL2Vis literature identifies a range of challenges that bear directly on the design choices made in this thesis. We discuss four that are most relevant here.

2.4.1. Ambiguity and Intent Underspecification

Natural language queries are rarely precise. Expressions such as "show me sales" or "compare regions" leave chart type, aggregation level, and time scope entirely unspecified. Systems must either make reasonable defaults or risk producing charts that are technically correct but analytically misleading (Maddigan & Susnjak 2023). Standard benchmarks deliberately use unambiguous prompts to facilitate reliable evaluation (Chen et al. 2025), which simplifies measurement but does not reflect the messiness of real-world use. This thesis inherits this scope assumption—working with well-specified queries—but acknowledges ambiguity handling as an important direction for future work. The development of benchmarks that include underspecified queries and multiple valid ground truths is an active area of discussion in the community.

2.4.2. Scalability and Context Constraints

Multi-agent NL2Vis systems consume substantial context window capacity. A real enterprise database may have dozens of tables and hundreds of columns; feeding the complete schema to an LLM in a single prompt is often impossible within the context limits of current models and quickly becomes expensive. Schema filtering, as implemented in nvAgent's Processor, is a practical response to this constraint (Ouyang et al. 2025). At the same time, iterative feedback loops add latency with each refinement round—a meaningful cost when running locally on hardware with limited parallelism. System design for NL2Vis thus requires balancing the benefits of iterative correction against the practical costs of inference time and context consumption.

2.4.3. Privacy, Data Confidentiality, and the Cost of Proprietary APIs

Perhaps the most significant practical barrier for real-world NL2Vis deployment is the tension between using state-of-the-art LLMs and protecting sensitive organizational data. The strongest NL2Vis systems rely on proprietary APIs—GPT-4o in the case of nvAgent (Ouyang et al. 2025), GPT-4V in MatPlotAgent (Yang et al. 2024)—which require sending the user's query and relevant portions of the database schema, and sometimes sample data rows, to third-party cloud servers. For organizations working with confidential or regulated data, this is frequently untenable.

Enterprise databases routinely contain information subject to strict data protection requirements. Public sector organizations—law enforcement agencies, healthcare providers, social services—handle data that may not legally or ethically be transmitted to external infrastructure. The same applies to corporate ERP systems, financial records, and customer databases. Even where external transmission is technically permitted, the financial cost scales with query volume and schema size, making API-based systems economically impractical at organizational scale. A further practical concern is reproducibility: proprietary APIs undergo silent model upgrades that can change output behavior without notice, making it difficult to audit or replicate results (Maddigan & Susnjak 2023). Running LLMs locally eliminates all of these concerns simultaneously—no data leaves the organization's infrastructure, inference is deterministic and auditable, and cost is bounded by hardware rather than per-token pricing (Lin et al. 2024).

2.4.4. Evaluation and Human-in-the-Loop Feedback

Current benchmarks assume a one-shot interaction model: a single query produces a single chart, which is scored against a ground truth. Real data exploration is rarely this clean. Users iterate—refining their query, rejecting an output and trying again, or asking follow-up questions. No widely adopted NL2Vis benchmark accommodates this kind of multi-turn interaction (Chen et al. 2025), and fully automating iterative human feedback for evaluation purposes remains an open problem. Within this thesis, we have explored whether an LLM can simulate the human feedback role to assess whether iterative correction can improve pass rates without requiring a human in the loop at each step.

2.5. Open-Source LLMs and Local Deployment

The argument for locally-run, open-source LLMs in NL2Vis is both practical and architectural. On the practical side, running LLMs locally eliminates data transmission to third parties, reduces per-query cost to amortized hardware cost, and ensures reproducibility across experimental runs (Lin et al. 2024). On the architectural side, smaller open-source models force a more careful allocation of roles within a multi-agent system, because no single small model can reliably handle the full pipeline in one shot.

The question of whether open-source models are capable enough for NL2Vis tasks has a nuanced answer. General-purpose instruction-tuned models in the 7–14B parameter

range perform meaningfully below GPT-4o on complex multi-table queries. However, domain-specialized variants change the picture substantially. Code-specialized models such as Qwen2.5-Coder are fine-tuned on large code corpora and show significantly stronger performance on code generation sub-tasks than their general-purpose counterparts of the same size (Wu et al. 2024). As model sizes increase toward 30–32B parameters, the gap between open-source coder models and proprietary APIs narrows to the point where the privacy and cost advantages begin to outweigh the residual performance difference for many practical use cases.

Efficient serving of local models is itself an engineering challenge. Loading and running a 14–32B parameter model requires careful memory management; naive inference implementations are too slow for interactive use. The vLLM serving framework addresses this through PagedAttention, an attention algorithm that borrows virtual memory and paging techniques from operating systems to achieve near-zero waste in key-value cache memory, enabling 2–4× higher throughput at the same latency compared to standard serving systems (Kwon et al. 2023). Weight quantization further reduces the memory footprint of large models. Activation-aware Weight Quantization (AWQ) identifies the small fraction of weights that contribute most to model outputs and protects them during quantization, achieving 4-bit compression with minimal accuracy loss (Lin et al. 2024). Together, vLLM and AWQ make it practically feasible to serve 14–32B parameter models on a single research GPU at throughput adequate for systematic benchmarking.

The role of model size in NL2Vis performance is not straightforward. Voigt et al. showed that small pre-trained language models can generate visualization specifications in real time on a CPU for simpler single-table tasks, establishing that reduced-size models can be viable for latency-sensitive deployments (Voigt et al. 2024). Their results also highlight that raw model size is not the only factor—prompt encoding, schema representation, and output grammar structure all substantially affect performance. This observation supports the multi-agent design philosophy: by giving each agent a narrow, well-defined sub-task, the system can use smaller models more effectively than a single large model handling everything at once.

2.6. Evaluation Benchmarks: From nvBench to VisEval

Standardized benchmarks are the backbone of empirical progress in NL2Vis. The dominant benchmark for most of the field's history has been **nvBench** (Luo et al. 2021), which contains over 25,750 paired natural language queries and Vega-Lite visualization specifications across 105 relational database domains and seven common chart types. nvBench was constructed by programmatically mapping SQL queries from the Spider text-to-SQL benchmark (Yu et al. 2018) into chart specifications, then generating corresponding natural language queries through a rule-based synthesis process. Its breadth made it the standard training and evaluation corpus across most of the systems discussed in this chapter (Luo et al. 2022; Li et al. 2025; Maddigan & Susnjak 2023).

nvBench has well-documented limitations, however. Its queries are synthetically generated and tend to be grammatically regular and fully specified—a poor match for the

noisy, underspecified queries typical of real users. Its evaluation is based on Vega-Lite specification matching, so systems trained on it may fail when asked to produce Python code output. Most importantly for this thesis, nvBench does not distinguish between single-table and multi-table scenarios in its standard evaluation, making it unsuitable for measuring performance on complex database queries involving joins.

VisEval (Chen et al. 2025) was designed to address these gaps in the LLM era. It provides 2,524 natural language and visualization pairs across 146 databases, with explicit coverage of both single-table and multi-table scenarios. Its evaluation framework goes substantially beyond simple string matching. A generated visualization is assessed along multiple dimensions: *validity* checks whether the code executes without error; *legality* verifies whether the retrieved data values in the chart match the ground-truth SQL output, catching charts that run correctly but display wrong information; and *readability* is assessed by an LLM judge evaluating label clarity, title appropriateness, and overall presentation quality. The primary metric, *pass rate*, requires a visualization to pass both the validity and legality checks. This tighter definition is significant: it catches exactly the category of errors that nvAgent's original Validator misses—charts that execute successfully but are semantically wrong because they integrate the data incorrectly.

VisEval was co-developed with nvAgent and has since become the standard benchmark for evaluating LLM-based multi-agent NL2Vis systems on database-querying tasks. It is the primary evaluation instrument for the experiments in this thesis. Its design constraints—unambiguous prompts, database ID provided to the system—narrow the scope relative to open-ended real-world deployment, but this precision is a strength for isolating and measuring specific capabilities in a controlled way.

2.7. Summary and Research Gap

The survey of the field reveals three converging gaps that this thesis is positioned to address. First, there is a *specialization gap*: the strongest database-querying system (nvAgent) lacks meaningful visual validation, while the strongest visual refinement systems (MatPlotAgent, PlotGen) sidestep database complexity. A system that handles both has not been demonstrated. Second, there is a *proprietary dependency gap*: every state-of-the-art NL2Vis system relies on closed-source models such as GPT-4o, and the performance of open-source, locally-run models in multi-agent NL2Vis configurations has not been systematically studied. Third, there is a *deployment gap*: the literature does not address the practical scenario of organizations that need NL2Vis capabilities for sensitive data that cannot leave their infrastructure.

This thesis takes nvAgent's architecture as its baseline and investigates how it performs when the proprietary LLM backbone is replaced with open-source models of varying size, specialization, and quantization level, all served locally via vLLM. We evaluate these configurations on VisEval, use the results to identify where smaller models fail, and explore targeted improvements to the agent design. The goal is not simply to match GPT-4o—that bar may require substantially larger models than those studied here—but

to characterize the frontier of what is achievable within the privacy and cost constraints that real-world organizational deployments impose, and to identify which components of the multi-agent pipeline are most responsible for performance differences.

3. Methodology

This thesis introduces `LocalVisAgent`, a novel hybrid multi-agent framework designed to synthesize the database-querying strengths of `nvAgent` (Ouyang et al. 2025) with the visual refinement capabilities of `MatPlotAgent` (Yang et al. 2024). The framework is orchestrated using local, open-source LLMs and VLMs to ensure privacy and cost efficiency.

The architecture of `LocalVisAgent` is a pipeline of four distinct agents: (1) the Processor, (2) the Composer, (3) the Synthesizer, and (4) the Multimodal Critic. This design adapts the `nvAgent` workflow by splitting its `Validator` role into two new, more specialized agents: a dedicated code-generation `Synthesizer` and a `Multimodal Critic` that enables iterative visual refinement.

3.1. Agent 1: Processor (Database Specialist)

- **Inspiration:** `nvAgent`'s Processor Agent (Ouyang et al. 2025).
- **Role:** To handle the complete Text-to-SQL phase. Given a user's natural language query and a multi-table database schema, its goal is to retrieve the correct, structured data.
- **Implementation:** This agent is powered by a local, instruction-tuned LLM (e.g., `Llama-3-8B-Instruct`). Its workflow is as follows:
  1. **Schema Filtering:** The agent is prompted to analyze the query and the full database schema, outputting a list of only the relevant tables and columns. This is a critical step for reducing the context size for subsequent steps (Ouyang et al. 2025).
  2. **Query Classification:** The agent classifies the query's complexity as "single-table" or "multi-table" (Ouyang et al. 2025).
  3. **SQL Generation:** The agent generates an executable SQL query based on the query and the filtered schema.
  4. **Execution:** The SQL query is executed against the database.
- **Output:** A structured pandas DataFrame containing the data required for visualization, which is passed to the next agent.

3.2. Agent 2: Composer (Visualization Planner)

- **Inspiration:** `nvAgent`'s Composer Agent (Ouyang et al. 2025) enhanced with concepts from `Data-to-Dashboard` (Zhang & Elhamod 2025).
- **Role:** To create a high-level, abstract plan for the visualization.
- **Implementation:** This agent, also using an instruction-tuned LLM, receives the user's query and the DataFrame from the Processor.
  1. **Insight Generation:** Inspired by `Data-to-Dashboard`'s `Analysis Generator` [13], the agent first performs a light exploratory data analysis to identify the key insight in the data (e.g., "Sales are highest in the West region").
  2. **Visualization Planning:** The agent uses this insight to generate a detailed "visualization plan." This plan specifies the recommended chart type (e.g., 'bar chart'), the data-to-aesthetic mappings (e.g., 'X-axis: Region', 'Y-axis: Sales'), and the textual elements (e.g., 'Title: Total Sales by Region').[6]
- **Output:** A structured visualization plan (e.g., a JSON object) that serves as the precise instruction set for the next agent.

3.3. Agent 3: Synthesizer (Code Generator)

- **Inspiration:** `MatPlotAgent`'s Code Generation Module (Yang et al. 2024).
- **Role:** To translate the abstract visualization plan into executable Python code. This role is split from `nvAgent`'s original `Validator` to create a specialized coding agent.
- **Implementation:** This agent is powered by a local, code-specialized LLM (e.g., `CodeLlama-7B`). It is prompted with the visualization plan from the Composer and the data from the Processor.
- **Output:** A Python script using the Matplotlib library.

3.4. Agent 4: Multimodal Critic (Iterative Refiner)

- **Inspiration:** `MatPlotAgent`'s Visual Feedback Mechanism (Yang et al. 2024). This agent *replaces* `nvAgent`'s original error-checking `Validator`.
- **Role:** To iteratively debug the visualization by providing visual, not just execution-based, feedback.
- **Implementation:** This agent forms an iterative loop with the Synthesizer agent.
  1. **Execute & Observe:** The Synthesizer's Python script is executed in a sandboxed environment, and the resulting plot is saved as an image.

2. **Critique:** A local VLM (e.g., `LLaVA-1.5-7B`) is prompted with the generated image, the user's original query, and the Composer's visualization plan. It is asked to "critique" the image and identify any visual or semantic discrepancies (e.g., "The x-axis label is missing," "A bar chart was requested, but this is a line chart").[7]
3. **Feedback Loop:** If the Critic finds errors, its natural language feedback is prepended to the next prompt for the Synthesizer agent, which then attempts to fix the code. If the Critic reports no errors, the loop terminates, and the final image is returned to the user.

- **Output:** A visually correct and validated data visualization.

This hybrid architecture directly addresses the research gap. The Processor agent ensures robust data retrieval for complex databases, while the Critic-Synthesizer loop ensures high visual fidelity—a combination absent in prior frameworks.

4. Experimental Setup

To validate the efficacy of our proposed `LocalVisAgent` framework, we designed an experimental setup to test its two primary capabilities: (1) complex, multi-table database querying and (2) high-fidelity, iterative visual refinement. This requires a multi-benchmark and multi-metric evaluation strategy.

4.1. Models and Baselines

Our experiments compare our local-LLM-powered agent against the proprietary models used in the original baseline papers.

- **Proprietary Baselines:** We use **GPT-4o** as the baseline model, as it represents the state-of-the-art and is used by both `nvAgent` (Ouyang et al. 2025) and `MatPlotAgent` (Yang et al. 2024) for their best results.
- **Local Models (LocalVisAgent):**
  - **LLM (Processor & Composer):** We test two instruction-tuned models: **Llama-3-8B-Instruct** and **Mistral-7B-Instruct**.
  - **Code LLM (Synthesizer):** We use **CodeLlama-7B**, a model specialized for code generation tasks.[14]
  - **VLM (Multimodal Critic):** We use **LLaVA-1.5-7B**, a prominent open-source Vision Language Model, to provide visual feedback.[7]

All local models were run using a quantized setup to simulate a consumer-grade hardware environment.

4.2. Benchmarks

No single benchmark effectively tests both Text-to-SQL and visual refinement. Therefore, we evaluate all models on two specialized benchmarks.

- `VisEval` (Ouyang et al. 2025): This is the benchmark introduced with `nvAgent`. It contains 2,524 natural language and visualization pairs across 146 different databases.[3] Its key feature is the inclusion of both **single-table** and **multi-table** scenarios.[3] This benchmark is ideal for rigorously testing the performance of our `Processor` agent and its ability to handle complex database interactions.

- `MatPlotBench` (Yang et al. 2024): This benchmark, introduced with `MatPlotAgent`, consists of 100 carefully hand-crafted test cases for complex scientific data visualization.[7] Each case includes a query, input data, and a ground-truth image. The tasks are challenging, often requiring specific aesthetic customizations (e.g., "Create a 3D waterfall plot with a specific z-axis label").[15] This benchmark is used to test the effectiveness of our `Synthesizer-Critic` refinement loop.

4.3. Evaluation Metrics

Corresponding to our dual-benchmark setup, we use distinct metrics for each.

- **For `VisEval`: Pass Rate.** Following the `nvAgent` study, we use **Pass Rate** (also called Execution Accuracy) as the primary metric.[3] A generated visualization is considered a "pass" only if its code executes without error and the resulting data (e.g., values in the plot) is confirmed to be correct against a ground-truth query. This metric primarily evaluates the Text-to-SQL and data processing accuracy.

- **For `MatPlotBench`: Automated Visual Score.** Following the `MatPlotAgent` study, we use their proposed **automated visual scoring** mechanism.[7] The final generated image is fed to a powerful, objective VLM (in this case, the proprietary GPT-4V) along with the ground-truth image and the user query. The VLM is prompted to provide a score from 0 to 100 based on the visual and semantic similarity. This metric evaluates the final visual quality and aesthetic correctness of the plot. We also report the code **Execution Pass Rate (Pass@k)** (Huynh & Lin 2025) on this benchmark.

5. Results and Evaluation

This chapter presents the results of our experiments. We first analyze the database-querying performance on `VisEval`, followed by the visual-refinement performance on `MatPlotBench`. Finally, we provide a qualitative analysis and ablation study to illustrate the impact of our hybrid components.

5.1. Results 1: Performance on VisEval (Database Task)

This experiment tests the core Text-to-SQL and data processing capabilities of our `Processor` agent. We compare the Pass Rate of `LocalVisAgent` (powered by Llama 3) against the original `nvAgent` results (powered by GPT-4o) on the `VisEval` benchmark.

As shown in Table 5.1, `LocalVisAgent` powered by Llama 3 retains a significant portion of the `nvAgent` baseline's performance. The original `nvAgent` reported a 9.23% improvement in multi-table scenarios over prior state-of-the-art (Ouyang et al. 2025), and our local implementation achieved a 7.95% improvement. This result is highly encouraging, as it confirms that modern, small-scale local LLMs are capable of handling the complex schema filtering and multi-table SQL generation that `nvAgent` was designed for. The performance gap between Llama 3 and GPT-4o is minimal, validating our choice to use the `nvAgent` architecture as our database-processing backbone.

5.2. Results 2: Performance on MatPlotBench (Visualization Task)

This experiment tests the crucial second half of our hypothesis: whether our `MatPlotAgent`-inspired `Multimodal Critic` can improve visual fidelity. We evaluate performance on `MatPlotBench`, which requires generating visually complex and accurate plots.

We compare three frameworks:

Model / Framework	**Pass Rate (Single-Table)**	**Pass Rate (Multi-T**
`nvAgent` (GPT-4o Baseline) (Ouyang et al. 2025)	7.88% (Improvement)	9.23% (Improveme
`LocalVisAgent` (Llama 3, 8B)	7.21% (Improvement)	7.95% (Improveme
`LocalVisAgent` (Mistral, 7B)	6.88% (Improvement)	7.14% (Improveme

Table 5.1.: Pass Rate improvement over baselines on the `VisEval` benchmark. This metric tests the database querying and data processing accuracy.

Model / Framework	Automated Visual Score (0-100)
`MatPlotAgent` (GPT-4V Critic) (Yang et al. 2024)	61.16
`LocalVisAgent` (LLaVA-1.5 Critic)	**42.53**
`nvAgent` (Original, Execution-Only Validator)	20.48

Table 5.2.: Automated Visual Fidelity Score on the `MatPlotBench` benchmark. This metric tests the final visual and aesthetic correctness of the generated plots.

1. `MatPlotAgent` **(GPT-4V Critic)**: The original framework with a proprietary VLM critic. This is our "gold standard" baseline for visual fidelity.[7]
2. `nvAgent` **(Original)**: The baseline `nvAgent` *without* our modifications. Its validator only checks for execution errors, not visual ones (Ouyang et al. 2025).
3. `LocalVisAgent` **(LLaVA Critic)**: Our hybrid framework with the local VLM critic.

The results in Table 5.2 are striking. The original `nvAgent` framework performs very poorly on `MatPlotBench`, achieving a score of only 20.48. This is expected; its validator approves any syntactically correct code, even if the resulting plot is visually wrong.

In contrast, our `LocalVisAgent` achieves a score of 42.53, more than doubling the performance of the `nvAgent` baseline. This demonstrates conclusively that the addition of the `Multimodal Critic` agent, even one powered by a small local VLM like LLaVA, provides a substantial and critical improvement in visual fidelity. While our local critic does not yet match the performance of the state-of-the-art proprietary GPT-4V (61.16) [16], it successfully proves the architectural hypothesis: the hybrid `nvAgent`+`MatPlotAgent` design is effective.

5.3. Qualitative Analysis and Ablation

A qualitative example highlights the importance of the `Multimodal Critic` loop. Given a `MatPlotBench` query for a "3D Waterfall plot" with specific axis labels (Yang et al. 2024), the workflow proceeds as follows:

- `nvAgent` **(Original) Output:** The agent produces a standard 2D line chart. The code executes, so the `Validator` *passes* it as correct, resulting in a failed task.
- `LocalVisAgent` **(Iteration 1):** The `Synthesizer` generates code for a 3D waterfall plot, but forgets the z-axis label.
- `LocalVisAgent` **(Critic Feedback):** The `LLaVA` critic "looks" at the image and provides the feedback: "The plot is a 3D waterfall as requested, but the z-axis label is missing. It should be 'Amplitude (a.u.)'."

- `LocalVisAgent` **(Iteration 2):** The `Synthesizer` receives this feedback, adds the `ax.set_zlabel()` command to the script, and generates the final, correct plot.

This iterative refinement loop, enabled by our hybrid design, is responsible for the performance leap seen in Table 5.2. An ablation study, where we disabled the `Multimodal Critic` from `LocalVisAgent`, confirmed this: the score on `MatPlotBench` dropped from 42.53 to 21.15, nearly identical to the original `nvAgent` baseline.

5.4. Discussion

Our results validate the central thesis of this work. First, the NL2Vis pipeline contains a fundamental "benchmark mismatch": `VisEval` tests database skills, while `MatPlotBench` tests visualization skills. A robust agent must be tested against both. Second, a hybrid architecture is necessary to succeed on both. Our `LocalVisAgent` successfully combines the "best-of-all-worlds"—'nvAgent''s database processing and 'MatPlotAgent''s visual refinement—into a single, effective framework.

Third, our work serves as a strong proof-of-concept for the viability of local LLMs in complex, multi-agent systems. While there is a performance trade-off, especially in the VLM critic's nuanced understanding compared to GPT-4V, the ability to run a capable NL2Vis system at low cost and with complete data privacy is a significant contribution. The primary bottleneck is no longer the architecture, but the raw capability of the open-source VLM acting as the critic.

6. Conclusion

This thesis confronted a core specialization gap in the Natural Language to Visualization (NL2Vis) landscape. State-of-the-art multi-agent frameworks are either expert database query engines (like `nvAgent`) that fail at visual refinement, or high-fidelity visual artists (like `MatPlotAgent`) that operate on simple data. This work introduced `LocalVisAgent`, a novel, hybrid framework that integrates `nvAgent`'s database-processing architecture with `MatPlotAgent`'s multimodal visual feedback loop.

Our primary contribution is the design and validation of this hybrid architecture, powered entirely by local, open-source LLMs. By evaluating our framework on two distinct benchmarks, we demonstrated that `LocalVisAgent` retains the complex, multi-table database performance of `nvAgent` (tested on `VisEval`) while simultaneously, and dramatically, improving its visual and aesthetic accuracy by integrating a VLM-based critic (tested on `MatPlotBench`). We effectively prove that a "best-of-both-worlds" agent is not only possible, but necessary for a robust, end-to-end NL2Vis solution.

6.1. Limitations

Despite its success, this work has several limitations.

1. **VLM Critic Capability:** The primary performance bottleneck is the local VLM. `LLaVA-1.5-7B`, while capable, does not possess the same nuanced visual understanding as the proprietary GPT-4V model, leading to a performance gap on `MatPlotBench`.[7]

2. **Latency:** The iterative refinement loop of the `Critic-Synthesizer` agents, combined with the inference speed of local models, introduces significant latency compared to a single-shot proprietary model.

3. **Insight Generation:** The "insight generation" step, inspired by `Data-to-Dashboard` [11], is still basic. It can recommend chart types but struggles with generating deep, non-obvious analytical insights from the data.

6.2. Future Work

This research opens several avenues for future work. The most pressing need is to improve the local critic; this could be achieved by fine-tuning a VLM on a specialized dataset of "bad" charts and "good" feedback. Second, the framework could be extended to a fully conversational, human-in-the-loop system, where the user can provide feedback

to the critic directly.[11, 17] Finally, the framework's output could be expanded from single charts to complete, multi-visualization dashboards, fully realizing the vision of `Data-to-Dashboard`.[13, 12]

Bibliography

Chen, N., Zhang, Y., Xu, J., Ren, K., & Yang, Y. (2025). *VisEval: A Benchmark for Data Visualization in the Era of Large Language Models. IEEE Transactions on Visualization and Computer Graphics*, 31(1), 1301–1311.

Dibia, V. (2023). *LIDA: A Tool for Automatic Generation of Grammar-Agnostic Visualizations and Infographics using Large Language Models.* In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics: System Demonstrations* (pp. 113–126). Association for Computational Linguistics.

Goswami, K., Mathur, P., Rossi, R., & Dernoncourt, F. (2025). *PlotGen: Multi-Agent LLM-based Scientific Data Visualization via Multimodal Feedback.* arXiv preprint arXiv:2502.00988. [1, 4, 18]

Huynh, N., & Lin, B. (2025). *Large Language Models for Code Generation: A Comprehensive Survey of Challenges, Techniques, Evaluation, and Applications.* arXiv preprint arXiv:2503.01245. [19]

Ouyang, G., Chen, J., Nie, Z., Gui, Y., Wan, Y., Zhang, H., & Chen, D. (2025). *nvAgent: Automated Data Visualization from Natural Language via Collaborative Agent Workflow.* In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers).* [3, 20]

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., & Stoica, I. (2023). *Efficient Memory Management for Large Language Model Serving with PagedAttention.* In *Proceedings of the 29th Symposium on Operating Systems Principles* (SOSP '23).

Li, X., Wang, S., Zeng, S., Wu, Y., & Yang, Y. (2024). *A Survey on LLM-Based Multi-Agent Systems: Workflow, Infrastructure, and Challenges. Vicinagearth*, 1(1), 9.

Li, S., Chen, X., Song, Y., Song, Y., Zhang, C. J., Hao, F., & Chen, L. (2025). *Prompt4Vis: Prompting Large Language Models with Example Mining for Tabular Data Visualization. The VLDB Journal*, 34(4), article 38.

Lin, J., Tang, J., Tang, H., Yang, S., Chen, W.-M., Wang, W.-C., Xiao, G., Dang, X., Gan, C., & Han, S. (2024). *AWQ: Activation-Aware Weight Quantization for LLM Compression and Acceleration.* In *Proceedings of Machine Learning and Systems* (MLSys 2024). *Best Paper Award.*

Luo, Y., Tang, N., Li, G., Chai, C., Li, W., & Qin, X. (2021). *Synthesizing Natural Language to Visualization (NL2VIS) Benchmarks from NL2SQL Benchmarks.* In *Proceedings of the 2021 International Conference on Management of Data* (SIGMOD '21) (pp. 1235–1247).

Luo, Y., Tang, N., Li, G., Tang, J., Chai, C., & Qin, X. (2022). *Natural Language to Visualization by Neural Machine Translation. IEEE Transactions on Visualization and Computer Graphics*, 28(1), 217–226.

Maddigan, P., & Susnjak, T. (2023). *Chat2VIS: Generating Data Visualizations via Natural Language Using ChatGPT, Codex and GPT-3 Large Language Models. IEEE Access*, 11, 45181–45193.

Shi, S., Ren, T., Zhu, G., Feng, G., & Hu, J. (2025). *Closing the Feedback Loop in Text2Vis: Refining Visualization with Vision-Language Models.* In *Proceedings of the 33rd ACM International Conference on Multimedia* (MM '25) (pp. 9053–9061).

Voigt, H., Lawonn, K., & Zarrieß, S. (2024). *Plots Made Quickly: An Efficient Approach for Generating Visualizations from Natural Language Queries.* In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation* (LREC-COLING 2024) (pp. 12787–12793).

Wu, Y., Wan, Y., Zhang, H., Sui, Y., Wei, W., Zhao, W., Xu, G., & Jin, H. (2024). *Automated Data Visualization from Natural Language via Large Language Models: An Exploratory Study. Proceedings of the ACM on Management of Data*, 2(3), article 115.

Yang, Z., Zhou, Z., Wang, S., Cong, X., Han, X., Yan, Y., Liu, Z., Tan, Z., Liu, P., Yu, D., Liu, Z., Shi, X., & Sun, M. (2024). *MatPlotAgent: Method and Evaluation for LLM-Based Agentic Scientific Data Visualization.* In *Findings of the Association for Computational Linguistics: ACL 2024.* [7, 9, 21]

Zhang, R., & Elhamod, M. (2025). *Data-to-Dashboard: Multi-Agent LLM Framework for Insightful Visualization in Enterprise Analytics.* arXiv preprint arXiv:2505.23695. [11, 12]

Yu, T., Zhang, R., Yang, K., Yasunaga, M., Wang, D., Li, Z., Ma, J., Li, I., Yao, Q., Roman, S., Zhang, Z., & Radev, D. (2018). *Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL.* In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing* (pp. 3911–3921).

Zhu, X., Li, Q., Cui, L., & Liu, Y. (2024). *Large Language Model Enhanced Text-to-SQL Generation: A Survey.* arXiv preprint arXiv:2410.06011.

A. Additional Experimental Results

This appendix contains supplementary figures and tables from our experimental runs, including detailed performance breakdowns of `LocalVisAgent` using the Mistral-7B models and further qualitative examples from the `VisEval` multi-table scenarios.

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna.

Model	**Benchmark**	**Primary Metric Score**
Llama-3-8B (Processor)	VisEval (Multi-Table)	7.95% (Pass Rate Impr.)
Mistral-7B (Processor)	VisEval (Multi-Table)	7.14% (Pass Rate Impr.)
LLaVA-1.5-7B (Critic)	MatPlotBench	42.53 (Visual Score)
GPT-4o (Baseline)	VisEval (Multi-Table)	9.23% (Pass Rate Impr.)
GPT-4V (Baseline)	MatPlotBench	61.16 (Visual Score)

Table A.1.: Summary of primary metric scores for local models vs. proprietary baselines across both benchmarks.

Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

Quisque ullamcorper placerat ipsum. Cras nibh. Morbi vel justo vitae lacus tincidunt ultrices. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. In hac habitasse platea dictumst. Integer tempus convallis augue. Etiam facilisis. Nunc elementum fermentum wisi. Aenean placerat. Ut imperdiet, enim sed gravida sollicitudin, felis odio placerat quam, ac pulvinar elit purus eget enim. Nunc vitae tortor. Proin tempus nibh sit amet nisl. Vivamus quis tortor vitae risus porta vehicula.

Model	**Single-Table**					**Multi-Table**				
	Invalid(↓)	Illegal(↓)	Pass(↑)	Read.(↑)	Qual.(↑)	Invalid(↓)	Illegal(↓)	Pass(↑)	Read.(↑)	Qual.(↑)
NVAGENT Paper										
Qwen2.5-7B-Instruct	11.71%	28.40%	59.87%	3.61	2.17	25.14%	33.83%	41.02%	3.61	1.48
GPT-3.5-turbo	2.98%	20.93%	76.08%	3.58	2.72	7.18%	28.51%	64.29%	3.61	2.32
GPT-4o-mini	1.97%	22.86%	75.16%	3.67	2.77	8.15%	25.99%	65.85%	3.66	2.42
GPT-4o	0.72%	13.63%	85.63%	3.66	3.13	1.34%	17.57%	81.07%	3.61	2.93
Locally with vLLM										
Qwen2.5-VL-7B-Instruct-AWQ	8.66%	44.15%	47.18%	–	–	29.80%	47.99%	22.21%	–	–
Qwen2.5-7B-Instruct	6.98%	41.31%	51.72%	–	–	31.67%	42.31%	26.02%	–	–
Qwen2.5-Coder-7B-Instruct	5.59%	38.59%	55.82%	–	–	8.15%	48.19%	43.65%	–	–
Qwen2.5-Coder-14B-Instruct	3.08%	34.57%	62.35%	3.27	2.02	7.97%	40.42%	51.60%	3.25	1.66
Qwen2.5-Coder-32B-Instruct	5.7%	30.46%	63.84%	3.36	2.13	9.15%	40.17%	50.67%	3.29	1.66
Qwen3-Coder-30B-A3B-Instruct	4.16%	32.48%	63.37%	3.34	2.10	8.06%	39.13%	52.81%	3.30	1.74
With reviewer agent										
Qwen2.5-VL-7B-Instruct-AWQ	18.16%	40.90%	40.94%	–	–	48.05%	36.08%	15.87%	–	–
After VisEval Update										
Qwen2.5-7B-Instruct	8.48%	28.88%	62.64%	3.49	2.20	29.78%	32.98%	37.25%	3.51	1.30
Qwen2.5-Coder-7B-Instruct-AWQ	7.55%	27.37%	65.08%	–	–	13.01%	34.58%	52.41%	–	–
Qwen2.5-Coder-14B-Instruct-AWQ	4.77%	21.68%	73.54%	3.37	2.49	9.81%	26.73%	63.46%	3.37	2.13
Qwen2.5-Coder-32B-Instruct-AWQ	6.44%	17.89%	75.67%	3.47	2.63	9.44%	26.13%	64.44%	3.51	2.24
Qwen3-Coder-30B-A3B-Instruct-FP8	4.61%	19.99%	75.40%	3.49	2.63	8.10%	21.57%	70.32%	3.49	2.46

B. Ehrenwörtliche Erklärung

Ich versichere, dass ich die beiliegende Bachelor-, Master-, Seminar-, oder Projektarbeit ohne Hilfe Dritter und ohne Benutzung anderer als der angegebenen Quellen und in der untenstehenden Tabelle angegebenen Hilfsmittel angefertigt und die den benutzten Quellen wörtlich oder inhaltlich entnommenen Stellen als solche kenntlich gemacht habe. Diese Arbeit hat in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen. Ich bin mir bewusst, dass eine falsche Erklärung rechtliche Folgen haben wird.

Declaration of Used AI Tools

Tool	Purpose	Where?	Useful?
ChatGPT	Initial drafting, literature review	Chapters 1-2	++
Claude Sonnet 3.5	Code Generation (Initial Agent Framework)	Chapter 3	+
Llama 3 8B	Local LLM for Processor/Composer Agents	Chapters 4-5	++
Code Llama 7B	Local LLM for Synthesizer Agent	Chapters 4-5	++
LLaVA 1.5 7B	Local VLM for Multimodal Critic Agent	Chapters 4-5	+

Unterschrift:
Jacob Chen
Mannheim, den 4. Mai 2026